Increasing Network Security with eBPF

Assignment #1: State of the art

António Lanção (up202205416)
David Sá (up202303580)
Henrique Teixeira (up202306640)

Network Security 2026/27 · Faculty of Sciences, University of Porto

The problem: kernel extension is a trade-off

- LKMs: arbitrary code, full kernel privilege, exact kernel match
- Fixed subsystems: only the policies their authors anticipated
- eBPF: untrusted code, **proven safe before it runs**, attached kernel-wide

> eBPF is a **substrate**, not a product. Defense systems are built on top, and the substrate is itself attack surface.

Execution and safety model

Lifecycle

- Small fixed instruction set, 11 registers.
- Verifier proves safety statically, before JIT.
- JIT to native code; attach as event handler.
- Maps: the only shared, durable state.

The verifier checks

- Termination: DAG check, bounded loops.
- Memory safety: range tracking vs. packet bounds.
- Types: per-register type state.
- **Not** intent, **not** authorization: it is the TCB.

Verifier safety ≠ authorization to load. Loading is gated separately:
`CAP_BPF`/`CAP_PERFMON` (Linux 5.8), `unprivileged_bpf_disabled`, BPF tokens.

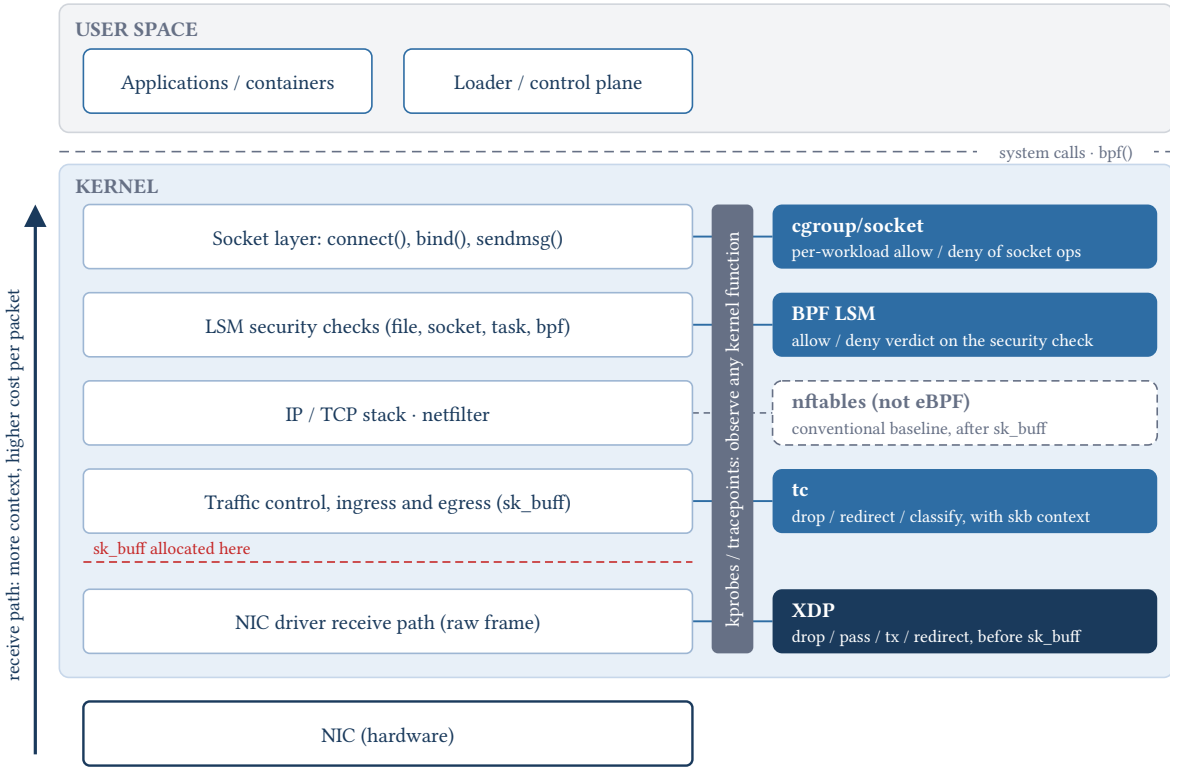
Hook points: the security decision

Hook	Input	Verdict	Cost
XDP	Raw frame, driver RX	Drop / pass / redirect	Lowest
tc	`sk_buff`, ingress/egress	Pass / drop / redirect	skb already paid
cgroup/socket	Workload, socket	Deny / restrict	No skb cost
kprobe/tracepoint	Function args	Observe only	Low
BPF LSM	Security check	Allow / deny	Function entry

Use case A (DDoS): XDP; identification needs header bytes only, and the target of defense is the machine itself.

Use case B (container control): socket/cgroup; identity-aware policy, BPF LSM as the generalization BPFContain takes.

Where each hook sits



- Receive path runs bottom to top
- XDP is the only network hook before `sk_buff` allocation
- nftables (our baseline) sits after it
- kprobes / tracepoints see every layer but only observe

Security systems: same substrate, different authors

System	Kind	Hooks	Overhead
Cilium	Production	XDP / tc / socket	None published
Tetragon	Production	kprobe / tracepoint / uprobe	None published
bpfbox	Research	LSM + probes	≈8.4% web throughput
BPFContain	Research	LSM + probes	< 16%, ≈ AppArmor

- Production (Cilium/Tetragon): label/pod identity, orchestration-native policy
- Research (bpfbox/BPFContain): confinement from the threat model up

> **Finding:** what separates them is **policy authorship** and the control plane's exposure, not capability. Overhead is only comparable in the research row.

eBPF itself is attack surface

Adversaries, from the literature

- Over-privileged container: ≈2.5% of active Docker Hub repos, no exploit needed
- Verifier/JIT bugs: ≥22 verifier bugs in 2021–22; JIT bug hijacks control flow
- Unsafe helpers: `bpf_probe_write_user`, map tamper via `bpf_map_get_fd_by_id`
- Policy bypass: process identity confusion, TOCTOU, fail-open on event loss
- Shared kernel is not a boundary: tracing ignores namespaces

The uncomfortable finding: the verifier is a hand-written, growing TCB; the literature argues it cannot scale as helpers multiply, and cross-container attacks work regardless of what it proved about the programs.

Assignment #2: two-layer eBPF firewall

Design

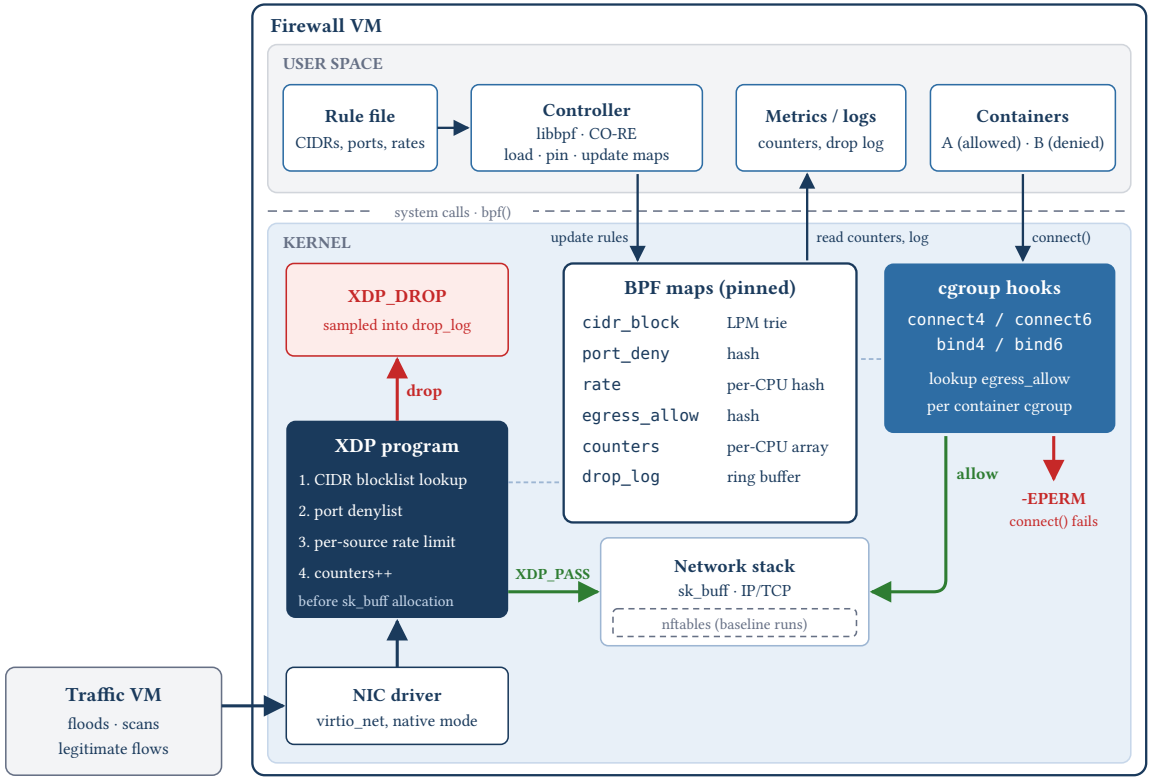
- XDP: CIDR blocklist (LPM trie), port denylist, per-source rate limit
- cgroup `connect`/`bind`: per-container egress, no host-port binds
- Controller: rules ⟶ maps at runtime; counters + drop log

Evaluation targets

- 100% expected verdicts on positive/negative tests
- Drop rate (Mpps) and CPU vs. nftables, same rules
- Zero legitimate drops under flood
- `CAP_BPF` + `CAP_NET_ADMIN` only; map-tamper test

> **Security property demonstrated:** correct enforcement of a defined policy, cheaper than nftables, with the privileges it adds measured.

Prototype flow



Conclusion

- The **hook is the security decision**: same bytecode, three defense regimes
- Production vs. research: policy authorship, not capability
- The verifier is a growing TCB; the kernel is a shared attack surface
- A#2: XDP + cgroup firewall, least privilege, every number published

Roadmap: deadline 6 December 2026

#	Milestone	Validation	Week
M1	XDP CIDR drop + counters	Attached; counters match sent	1
M2	Controller, cgroup hooks, drop log	Runtime rule changes apply	1–2
M3	Policy test suite	All +/− cases pass	2
M4	Benchmark + nftables baseline	Scripted, repeated runs	3–4
M5	Privilege audit + map tamper	`CAP_BPF` + `CAP_NET_ADMIN` only	4
M6	Reproducibility artefacts	Fresh VM reproduces a run	4

35 days from today; 5 working weeks; M6 leaves the final week free for the report and packaging (`TP1_group7_report_2.pdf`).

Questions?